

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI

BỘ MÔN TOÁN HỌC ỨNG DỤNG



GIÁO TRÌNH
CÁC MÔ HÌNH TÍNH TOÁN

Giảng viên: Nguyễn Văn Hưng

Sinh viên:MSSV:Lớp:

Hà nội 01-2026

CÁC YÊU CẦU ĐỐI VỚI MÔN HỌC

1. Sinh viên đi học đúng giờ. Sinh viên vào lớp sau khi giáo viên đã bắt đầu bài giảng sẽ **không được vào lớp**.

2. Sinh viên ghi bài và làm các ví dụ vào trong quyển in để tính điểm.

3. Lịch nộp ví dụ các buổi học và các bài tập sẽ theo lịch trên Microsoft Teams.

4. **Sinh viên cài Microsoft Teams trên điện thoại hoặc máy tính để lấy số liệu cho ví dụ, bài tập, bài kiểm tra, bài thi.**

5. **Trong giờ học tuyệt đối không sử dụng điện thoại. Nếu sử dụng điện thoại sẽ tính là sử dụng tài liệu.**

6. **Sẽ điểm danh đầu buổi học. Sinh viên vắng đầu buổi học sẽ tính vắng cả buổi.**

7. **Sinh viên nghỉ học chỉ khi có giấy phép của phòng Công tác sinh viên hoặc phòng Y tế.**

8. **Sinh viên nghỉ 7 – 9 tiết và trung bình điểm 1 hoặc 2 dưới 4 sẽ không đủ điều kiện dự thi Nghỉ trên 12 tiết sẽ không được thi.**

9. Số liệu các ví dụ của buổi học sẽ được gửi trên Microsoft Teams trước mỗi buổi học 30 phút. Sinh viên hoàn thành **điền các đáp số tương ứng vào các ô màu xanh lá cây và trình bày vào quyển ghi**. Sau khi kết thúc buổi học thì sẽ đóng các ví dụ và chấm điểm.

Nếu sinh viên không tuân thủ theo các quy định nêu trên khi làm bài thi sẽ nhận điểm 0.

CÁCH TÍNH ĐIỂM

- Điểm 1 chiếm 25% tổng điểm, là điểm bài tập các buổi, điểm bài tập chương và vở ghi.
- Điểm 2 chiếm 25% tổng điểm: là điểm của các bài kiểm tra.
- Điểm cuối kỳ chiếm 50% tổng điểm.

TÀI LIỆU THAM KHẢO

1. **Ngôn ngữ hình thức** – Nguyễn Văn Ba. Nhà xuất bản khoa học Kỹ thuật.
2. **Bài giảng môn học: Automata và ngôn ngữ hình thức** – Hồ Văn Quân. Đại học Bách Khoa Hồ Chí Minh.
3. **An Introduction to Formal Languages and Automata, Fifth Edition** – **Peter Linz**.

Các bài giảng của giáo sư, cho dù có đầy đủ, xúc tích đến đâu, có chứa chan tình yêu tri thức của bản thân giáo viên đến đâu, thì về thực chất, mà nói, đó chẳng qua cũng vẫn chỉ là chương trình, là những lời chỉ dẫn tuần tự để điều chỉnh trật tự nhận thức của sinh viên. Người nào chỉ biết ngồi nghe giáo sư giảng chữ bản thân mình trong lòng không cảm thấy khát khao học, thì có thể nói tất cả những điều người ấy nghe giảng ở trường đại học cũng sẽ chỉ như một tòa nhà xây trên cát mà thôi.

I.A. GONTCHAROV

GIỚI THIỆU VỀ MÔN HỌC CÁC MÔ HÌNH TÍNH TOÁN (AUTOMATA VÀ NGÔN NGỮ HÌNH THỨC)

I. Mở đầu

Trong Khoa học máy tính, câu hỏi cốt lõi không chỉ là *máy tính làm được gì*, mà còn là *có những bài toán nào về nguyên tắc không thể giải được*, và *những bài toán nào có thể giải được với tài nguyên hữu hạn*. Để trả lời những câu hỏi này, các nhà khoa học đã xây dựng nên các **mô hình tính toán (models of computation)**, tiêu biểu là **automata** và **ngôn ngữ hình thức (formal languages)**. Đây là nền tảng lý thuyết quan trọng bậc nhất của Khoa học máy tính hiện đại.

II. Lịch sử ra đời và phát triển

1. Nguồn gốc từ toán học và logic

Cuối thế kỷ XIX, toán học đối mặt với cuộc khủng hoảng nền tảng khi xuất hiện các nghịch lý trong lý thuyết tập hợp (Russell, Cantor). Điều này thúc đẩy sự phát triển của:

- Logic hình thức (Frege, Russell, Hilbert).
- Hình thức hóa khái niệm suy luận và chứng minh.

David Hilbert đặt ra chương trình Hilbert với mục tiêu:

“Mọi mệnh đề toán học đều có thể được quyết định bằng một thuật toán hữu hạn.”

Câu hỏi này trực tiếp dẫn tới việc nghiên cứu khái niệm thuật toán và tính toán.

2. Giai đoạn khởi đầu (1930 – 1940): Những câu hỏi về tính toán.

Hilbert nêu ra **Entscheidungsproblem**:

“Có tồn tại một thuật toán tổng quát để quyết định mọi mệnh đề logic đúng hay sai hay không?”

Để trả lời, cần trước hết định nghĩa chính xác thế nào là “*thuật toán*”.

Trong giai đoạn 1930 – 1936, ba mô hình tính toán quan trọng ra đời độc lập nhưng tương đương về sức mạnh:

- 1936 – **Alan Turing và máy Turing**: Đây là cột mốc quan trọng nhất. Turing đã đề xuất một mô hình máy lý thuyết có thể thực hiện bất kỳ thuật toán nào. Máy Turing chính là nền tảng cho kiến trúc máy tính hiện đại.
- **Alonzo Church và phép tính Lambda (λ – calculus)**: Cùng thời điểm, Church đưa ra một mô hình tính toán dựa trên hàm số, sau này trở thành nền tảng cho các ngôn ngữ lập trình hàm như Lisp, Haskell.
- **Gödel, Church** : Hàm đệ quy.

Kết quả nổi tiếng trong giai đoạn này:

- **Church–Turing thesis** cho rằng mọi quá trình tính toán hiệu quả đều có thể được mô phỏng bởi máy Turing.
- Không tồn tại thuật toán tổng quát cho mọi bài toán.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Entscheidungsproblem có lời giải âm.

Giai đoạn này đã đặt nền tảng lý thuyết cho toàn bộ khoa học máy tính hiện đại và từ đây, *lý thuyết tính toán (Theory of Computation)* chính thức hình thành.

3. Giai đoạn hình thành (1940 – 1960): Thần kinh học và ngôn ngữ học

- 1943 — *McCulloch & Pitts*: Đề xuất mô hình logic của các tế bào thần kinh, đặt viên gạch đầu tiên cho Máy tự động hữu hạn (Finite Automata).
- 1956 — *Noam Chomsky và phân cấp Chomsky*: Nhà ngôn ngữ học Chomsky đã phân loại các ngôn ngữ hình thức thành 4 loại (từ loại 3 đến loại 0). Đây là câu nổi quan trọng nhất kết hợp ngôn ngữ học với toán học và máy tính.
- *Stephen Kleene*: Giới thiệu khái niệm biểu thức chính quy (Regular Expressions) và chứng minh sự tương đương giữa chúng với máy tự động hữu hạn.

4. Giai đoạn hoàn thiện và mở rộng (1960 – nay)

- Các nhà khoa học như Stephen Cook và Richard Karp bắt đầu nghiên cứu về độ phức tạp tính toán (P vs NP), xác định xem máy tính có thể giải quyết các bài toán nhanh đến mức nào.
- Lý thuyết văn phạm phi ngữ cảnh được ứng dụng trực tiếp để thiết kế các trình biên dịch cho các ngôn ngữ như Algol, C.
- Phát triển automata ngăn xếp, automata tuyến tính, automata xác suất, automata song song.
- Mở rộng lý thuyết ngôn ngữ sang ngôn ngữ hình thức cho lập trình, đặc tả hệ thống, xác minh hình thức.

III. Ý nghĩa của các mô hình tính toán

1. Ý nghĩa lý thuyết

- Cung cấp định nghĩa chính xác cho khái niệm thuật toán và tính toán.
- Giúp phân loại các bài toán theo khả năng giải được và mức độ phức tạp.
- Là cơ sở cho các kết quả quan trọng như:
 - + Bài toán không quyết định được.
 - + Bài toán dừng.
 - + Các lớp độ phức tạp (P, NP, PSPACE, ...).

2. Ý nghĩa phương pháp luận

- Cho phép mô hình hóa các hệ thống phức tạp bằng các cấu trúc hình thức.
- Tạo ra ngôn ngữ chung để mô tả, phân tích và chứng minh tính đúng đắn của hệ thống.

IV. Ứng dụng của mô hình tính toán

1. Trong trình biên dịch và ngôn ngữ lập trình

Đây là ứng dụng thực tế nhất của mô hình tính toán:

- *Automata hữu hạn*: phân tích từ vựng (lexical analysis)
- *Văn phạm phi ngữ cảnh*: phân tích cú pháp (parsing)

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- *Ngôn ngữ hình thức*: định nghĩa cú pháp và ngữ nghĩa của ngôn ngữ lập trình.
- *Kiểm tra cú pháp và cấu trúc chương trình*.

2. *Xác minh và kiểm chứng hệ thống*

- Dùng máy tự động để chứng minh rằng các hệ thống quan trọng (như hệ thống điều khiển máy bay, phanh ABS) không bao giờ rơi vào trạng thái lỗi.
- Kiểm tra tính đúng đắn của : Phần mềm, giao thức mạng, hệ thống nhúng

3. *Trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên*

- Các mô hình văn phạm giúp máy tính hiểu cấu trúc câu của con người.
- Automata và grammar hỗ trợ: Phân tích câu, nhận dạng mẫu, xử lý chuỗi ký tự.

4. *An toàn thông tin và hệ thống*

- Phân tích luồng điều khiển.
- Phát hiện hành vi bất thường.
- Kiểm tra lỗi hỏng logic trong hệ thống.
- Kiểm tra mã độc.

V. *Các học phần liên quan*

Nếu bạn theo đuổi ngành Khoa học máy tính, bạn sẽ gặp các khái niệm này trong các môn học sau:

- *Lý thuyết tính toán (Theory of Computation)*: Môn học cốt lõi nghiên cứu về Automata, Ngôn ngữ hình thức và Máy Turing.
- *Ngôn ngữ lập trình & Trình biên dịch (Compilers)*: Cách chuyển đổi mã nguồn thành mã máy.
- *Cấu trúc dữ liệu và Giải thuật*: Liên quan đến độ phức tạp tính toán và hiệu quả của các mô hình.
- *Thiết kế hệ thống số (Digital Logic Design)*: Ứng dụng máy tự động hữu hạn vào phần cứng.
- *Trí tuệ nhân tạo (AI)*: Các mô hình Markov, mạng thần kinh đời đầu có gốc rễ từ lý thuyết Automata.
- *An toàn thông tin*: Sử dụng lý thuyết ngôn ngữ để phát hiện mã độc hoặc tấn công mạng.

VI. *Kết luận*

Các mô hình tính toán, đặc biệt là automata và ngôn ngữ hình thức, không chỉ là những khái niệm trừu tượng mang tính lý thuyết mà còn là xương sống của khoa học máy tính hiện đại. Chúng kết nối toán học, logic và lập trình, giúp con người hiểu rõ giới hạn của tính toán, thiết kế hệ thống hiệu quả và đảm bảo tính đúng đắn cho các ứng dụng quan trọng.

Nếu Khoa học máy tính là một ngôi nhà, thì Automata và Ngôn ngữ hình thức chính là **nền móng**. Bạn không cần nhìn thấy nó mỗi ngày khi ở trong nhà, nhưng không có nó, ngôi nhà không thể tồn tại.

CHAPTER I. NGÔN NGỮ VÀ BIỂU DIỄN NGÔN NGỮ

§1. KHÁI NIỆM VỀ NGÔN NGỮ

I. Mở đầu

Ngôn ngữ là phương tiện để giao tiếp. Việc giao tiếp có thể là giữa người với người; giữa người với máy; giữa máy với máy. Tất cả chúng ta đều quen thuộc với ngôn ngữ tự nhiên. Đó là ngôn ngữ để con người giao tiếp với nhau, ví dụ như ngôn ngữ tiếng Việt, ngôn ngữ tiếng Anh, ngôn ngữ tiếng Pháp, ... Từ điển định nghĩa ngôn ngữ một cách không chính thức như một hệ thống phù hợp để diễn đạt các ý tưởng, sự kiện hoặc khái niệm nhất định, bao gồm một tập hợp các ký hiệu và quy tắc để sử dụng chúng. Điều này cho chúng ta một ý tưởng trực quan về ngôn ngữ là gì, nhưng nó không đủ để làm định nghĩa cho việc nghiên cứu các ngôn ngữ chính thức. Đặc điểm của ngôn ngữ tự nhiên là quy tắc cú pháp thường rất phức tạp nhưng về mặt ngữ nghĩa thường thiếu chặt chẽ (ví dụ một câu có thể hiểu theo nhiều cách khác nhau tùy từng hoàn cảnh cụ thể).

Con người muốn giao tiếp với máy tính cũng phải thông qua ngôn ngữ. Tuy nhiên không thể sử dụng ngôn ngữ tự nhiên do tính thiếu chặt chẽ và đa nghĩa của ngôn ngữ tự nhiên. Ngôn ngữ này cần có cú pháp, ngữ nghĩa chặt chẽ, với một từ hay một câu thì nghĩa của nó là duy nhất không phụ thuộc ngữ cảnh. Ngôn ngữ như vậy được gọi là ngôn ngữ hình thức.

Con người muốn máy tính thực hiện công việc cần phải viết yêu cầu cho máy tính bằng ngôn ngữ mà máy có thể hiểu được. Việc viết yêu cầu cho máy là lập trình còn ngôn ngữ dùng để lập trình là ngôn ngữ lập trình. Ngôn ngữ lập trình cũng là ngôn ngữ hình thức.

Cả ngôn ngữ hình thức lẫn ngôn ngữ tự nhiên đều có thể xem như những tập các từ, tức là các xâu hữu hạn các phần tử của một bộ chữ cái cơ sở nào đó. Về mặt truyền thống, lý thuyết ngôn ngữ hình thức liên quan đến các đặc tả cú pháp của ngôn ngữ nhiều hơn là đến những vấn đề ngữ nghĩa. Một đặc tả về cú pháp của một ngôn ngữ có hữu hạn từ, ít nhất về nguyên tắc, có thể được cho bằng cách liệt kê các từ. Điều đó không thể áp dụng đối với các ngôn ngữ có vô hạn từ. Nhiệm vụ chính của lý thuyết ngôn ngữ hình thức là nghiên cứu các cách đặc tả hữu hạn của các ngôn ngữ vô hạn.

Lý thuyết tính toán cũng như của nhiều ngành khác nhau của nó, chẳng hạn mật mã học, có liên quan mật thiết với lý thuyết ngôn ngữ. Các tập vào và ra của một thiết bị tính toán có thể được xem như các ngôn ngữ và nói một cách sâu sắc hơn thì các mô hình tính toán có thể được đồng nhất với các lớp các đặc tả ngôn ngữ theo nghĩa mà trong bài giảng này chúng ta sẽ nêu chính xác hơn.

II. Một số khái niệm về ngôn ngữ

1. Bảng chữ và xâu

Định nghĩa 1.2.1.

Ví dụ 1.2.2.

- Bảng chữ tiếng Anh $\Sigma = \{a, b, c, \dots, z\}$.
- Bảng chữ tiếng Việt.
- Bảng chữ số thập phân $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- Bảng chữ số nhị phân $\Sigma = \{0, 1\}$.

Định nghĩa 1.2.3.

Quy ước.

- ◇ Các chữ cái của bảng chữ Σ được ký hiệu là $a, b, c, \dots$ (có thể thêm chỉ số).
- ◇ Các xâu được ký hiệu là $u, v, w, \dots$ (có thể thêm chỉ số).

Ví dụ 1.2.4.

- Từ “chinh” là một xâu trên bảng chữ tiếng Việt.
- Từ “rose” là một xâu trên bảng chữ tiếng Anh.
- Từ “010110” là một xâu trên bảng chữ nhị phân.
- Từ “2103147” là một xâu trên bảng chữ thập phân: $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Định nghĩa 1.2.5.

Ví dụ 1.2.6.

- $|chinh| = 5$.
- $|rose| = 4$.
- $|010110| = 6$.
- $|2103147| = 7$.

Định nghĩa 1.2.7.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Nhận xét. $|\lambda| = 0$.

Định nghĩa 1.2.8.

Ví dụ 1.2.9. Trong bảng chữ nhị phân $\Sigma = \{0,1\}$:



2. Các phép toán trên chuỗi

Định nghĩa 1.2.10.

Ví dụ 1.2.11.

Định nghĩa 1.2.12.

Ví dụ 1.2.13.



Định nghĩa 1.2.14.

Ví dụ 1.2.15.



A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success



Định nghĩa 1.2.14.

Ví dụ 1.2.17.



Định nghĩa 1.2.18.

Ví dụ 1.2.19. Xâu $u = \text{"able was I ere I saw elba"}$ có xâu đảo ngược là $u^R = \text{"able was I ere I saw elba"}$.

Đây là câu nói nổi tiếng của Napoleon Bonaparte khi bị lưu đày ra đảo Elba.

Nhận xét.

- ❖ Xâu rỗng λ là phần tử đơn vị đối với phép nhân ghép vì với mọi xâu u , ta có: $\lambda u = u\lambda = u$ nên xâu λ là tiền tố và hậu tố của mọi xâu u .
- ❖ Với mọi xâu u, v ta có: $|uv| = |vu| = |u| + |v|$.
- ❖ Với mọi xâu u và $n \in \mathbb{N}$ ta có $|u^n| = n|u|$.
- ❖ uv và vu có thể là hai xâu khác nhau.

III. Ngôn ngữ

1. Ngôn ngữ

Định nghĩa 1.3.1.

Ví dụ 1.3.2.

- Ngôn ngữ tiếng Việt là ngôn ngữ trên bảng chữ cái tiếng Việt.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Ngôn ngữ tiếng Anh là ngôn ngữ trên bảng chữ cái tiếng Anh.
- $L = \{0, 01, 101, 01001\}$ là ngôn ngữ trên bảng chữ nhị phân.
- $L = \Sigma^*$ là ngôn ngữ trên bảng chữ Σ .

Định nghĩa 1.3.3.

Chú ý: $\emptyset \neq \{\epsilon\}$.

Định nghĩa 1.3.4.

Ví dụ 1.3.5.

- Ngôn ngữ tiếng Việt là ngôn ngữ hữu hạn.
- $\emptyset, \{\epsilon\}$ là các ngôn ngữ hữu hạn.
- Ngôn ngữ trên bảng chữ thập phân (tập các số tự nhiên) là ngôn ngữ vô hạn.

2. Các phép toán trên ngôn ngữ

Định nghĩa 1.3.6. Cho hai ngôn ngữ L_1, L_2 cùng trên bảng chữ Σ .

- Hợp của hai ngôn ngữ L_1, L_2 trên bảng chữ Σ , ký hiệu là $L_1 \cup L_2$, là một ngôn ngữ trên bảng chữ Σ mà các từ thuộc vào một trong hai ngôn ngữ L_1, L_2 .
- Giao của hai ngôn ngữ L_1, L_2 trên bảng chữ Σ , ký hiệu là $L_1 \cap L_2$, là một ngôn ngữ trên bảng chữ Σ mà các từ thuộc vào cả hai ngôn ngữ L_1, L_2 .
- Phần bù của ngôn ngữ L_1 trên bảng chữ Σ , ký hiệu là $C_\Sigma(L)$, là một ngôn ngữ trên bảng chữ Σ , là tập hợp $\Sigma^* \setminus L_1$.

Ví dụ 1.3.7. Cho ngôn ngữ $L_1 = \{ab, ba, aba, baba\}$, $L_2 = \{ba, bab, bbaa, baba\}$ trên bảng chữ $\Sigma = \{a, b\}$. Khi đó:

$$L_1 \cup L_2 = \{ab, ba, aba, bab, baba, bbaa\}$$

$$L_1 \cap L_2 = \{ba, baba\}$$

Định nghĩa 1.3.8.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 1.3.9. Cho ngôn ngữ $L_1 = \{ab, ba\}$ trên bảng chữ $\Sigma_1 = \{a, b\}$, $L_2 = \{ac, cd\}$ trên bảng chữ $\Sigma_2 = \{a, c, d\}$. Khi đó:

Định nghĩa 1.3.10.

Ví dụ 1.3.11. Cho ngôn ngữ $L = \{aa, ab\}$ trên bảng chữ $\Sigma = \{a, b\}$. Khi đó:

Định nghĩa 1.3.12.

•

•

Nhận xét.

❖ $L^* = L^+ \cup \{\lambda\}$.

❖ Trong L^+ có thể chứa chuỗi λ nếu L chứa chuỗi λ .

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 1.3.13. Cho ngôn ngữ $L = \{0,1\}$ trên bảng chữ nhị phân $\Sigma = \{0,1\}$. Khi đó:

- $L^0 = \{\lambda\}$.
- $L^1 = \{0,1\}$.
- $L^2 = \{00,01,10,11\}$
- $L^3 = \{000,001,010,011,100,101,110,111\}$.
- L^n là tập các xâu nhị phân độ dài $n (n \geq 0)$.
- L^* là tập các xâu nhị phân, L^+ là tập các xâu nhị phân độ dài lớn hơn hoặc bằng 1.

Ví dụ 1.3.14. Cho bảng chữ $\Sigma = \{a\}$. Biểu diễn các ngôn ngữ sau:

- a) L_1 : tập các xâu trên Σ có độ dài lẻ.

- b) L_2 : tập các xâu trên Σ có độ dài chẵn ít nhất 2.

Chú ý. Cho ba ngôn ngữ cùng trên bảng chữ $\Sigma = \{a, b, c\}$:

$$L_1 = \{a^n b^n c^n | n \geq 0\}$$

$$L_2 = \{a^n b^n | n \geq 0\} \{c^n | n \geq 0\} = \{a^n b^n | n \geq 0\} \{c^m | m \geq 0\} = \{a^n b^n | n \geq 0\} \{c\}^*$$

$$L_3 = \{a^n | n \geq 0\} \{b^n | n \geq 0\} \{c^n | n \geq 0\} = \{a^n | n \geq 0\} \{b^m | m \geq 0\} \{c^k | k \geq 0\} \\ = \{a\}^* \{b\}^* \{c\}^*$$

là ba ngôn ngữ khác nhau. Khi đó với ba xâu $u_1 = aaabbbccc, u_2 = aaabbbcc, u_3 = abbbcc$ thì:

- ✓ Xâu u_1 thuộc vào cả ba ngôn ngữ L_1, L_2, L_3 .
- ✓ Xâu u_2 thuộc vào ngôn ngữ L_2, L_3 nhưng không thuộc vào L_1 .
- ✓ Xâu u_3 thuộc vào ngôn ngữ L_3 nhưng không thuộc vào L_1, L_2 .

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định nghĩa 1.3.15.

Ví dụ 1.3.16. Ngôn ngữ $L = \{\text{star}, \text{desserts}, \text{live}\}$ trên bảng chữ tiếng Anh có ngôn ngữ ngược là:

$$L^R = \{\text{rats}, \text{stressed}, \text{evil}\}$$

BÀI TẬP BUỔI 1

Bài tập xâu. Cho xâu $u =$

$v =$

Khi đó:

$$|u| =$$

$$|v| =$$

$$uu =$$

$$uu^R =$$

$$u^R u =$$

Xâu

Xâu

$$uv =$$

$$vu =$$

$$vvv =$$

Bài tập ngôn ngữ. Cho ngôn ngữ $L_1 = \{$

$L_2 = \{$

$\}$. Khi đó:

$$L_1 L_1 =$$

$$L_1 L_1^R =$$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$$L_1^R L_1 =$$

$$L_1 L_2 =$$

$$L_2 L_1 =$$

§2. KHÁI NIỆM VĂN PHẠM VÀ AUTOMATA

1. Biểu diễn ngôn ngữ

Một ngôn ngữ L trên bảng chữ cái Σ là một tập con của tập Σ^* . Vấn đề đặt ra là đối với một ngôn ngữ L có cách nào chỉ rõ một xâu có thuộc L hay không.

Ví dụ 2.1.1. Từ “bay” vừa là từ thuộc ngôn ngữ tiếng Việt, vừa thuộc ngôn ngữ tiếng Anh.

Đối với các ngôn ngữ hữu hạn, để biểu diễn chúng thì chúng ta chỉ cần liệt kê ra tất cả các xâu thuộc vào ngôn ngữ.

Ví dụ 2.1.2. Ngôn ngữ tiếng Việt là hữu hạn nên có thể liệt kê tất cả các từ tiếng Việt.

Đối với ngôn ngữ vô hạn không phức tạp, chúng ta có thể xác định các xâu bằng cách chỉ ra đặc điểm cốt yếu của mỗi xâu.

Ví dụ 2.1.3.

➤

➤

➤

Ví dụ 2.1.4. L là tập các xâu uv sao cho u là xâu gồm toàn ký tự a có số chẵn ký tự a , v là xâu gồm toàn ký tự b và có độ dài ít nhất 2.

Để nghiên cứu ngôn ngữ bằng toán học, chúng ta cần một phương pháp để mô tả chúng. Ngôn ngữ tự nhiên thường không chính xác và mơ hồ, vì vậy các mô tả ngôn ngữ thường không đủ. Ký hiệu tập hợp được sử dụng trong ví dụ 3.1.3 và 3.1.4 tuy phù hợp hơn, nhưng bị hạn chế. Để tiếp tục

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

chúng ta sẽ tìm hiểu một số cách định nghĩa ngôn ngữ mà nó hữu ích trong các trường hợp khác nhau. Ở đây chúng tôi sẽ giới thiệu một khái niệm phổ biến và mạnh mẽ, khái niệm về văn phạm.

II. Văn phạm

Văn phạm (theo ngôn ngữ học) là tập các quy tắc về cấu tạo từ và các quy tắc liên kết các từ thành câu. Văn phạm cho chúng ta biết liệu một câu cụ thể có được tạo ra đúng chuẩn hay không. Một quy tắc điển hình của ngữ pháp là “một câu bao gồm một chủ ngữ theo sau là một vị ngữ”. Chúng ta có thể viết điều này một cách đơn giản và rõ ràng hơn như sau:

<Câu> → <Chủ ngữ> <Vị ngữ>

Tuy nhiên điều này là không đủ để mô tả các câu trong thực tế. Chúng ta sẽ định nghĩa cho các cấu trúc mới được giới thiệu “Chủ ngữ” và “Vị ngữ”. Khi đó chúng ta sẽ thực hiện như sau:

<Chủ ngữ> → <Danh từ>

<Vị ngữ> → <Động từ>

và nếu chúng ta kết hợp các từ “Cây” và “Trời” với “mưa” và “ngủ”, khi đó theo ngữ pháp chúng ta biết được các câu “Cây mưa” và “Trời ngủ” được tạo thành đúng chuẩn. Nếu chúng ta có thể đưa ra một văn phạm hoàn chỉnh thì về mặt lý thuyết, mọi câu thích hợp đều có thể được giải thích theo cách này.

Định nghĩa 2.2.1

•

•

Định nghĩa 2.2.2.

•

•

•

•

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Các quy tắc sản xuất là thành phần cốt lõi của văn phạm. Các sản xuất xác định cách văn phạm biến đổi một xâu này thành một xâu khác và thông qua đó, chúng sẽ xác định một ngôn ngữ được sinh bởi văn phạm.

Định nghĩa 2.2.3.

•

•

Chú ý. Dấu * thể hiện số bước suy dẫn không xác định (có thể là 0) để có thể nhận được xâu y từ xâu x .

✓ Nếu trong tập sản xuất P có một loạt các sản xuất có cùng vế trái

$$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$$

thì các sản xuất này thường được viết gộp lại:

$$\alpha \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

Nhận xét. $\Rightarrow$ là quan hệ hai ngôi trên Σ^* còn $\Rightarrow^*$ là bao đóng bắc cầu, phản xạ trên Σ^* .

Bằng cách áp dụng các quy tắc sản xuất theo các cách khác nhau thì một văn phạm có thể tạo ra nhiều xâu. Tập hợp tất cả các xâu gồm các ký hiệu kết thúc như vậy là ngôn ngữ được định nghĩa hay sinh bởi văn phạm.

Định nghĩa 2.2.4.

•

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Quy ước.

- ◇ Các ký tự in hoa $A, B, C, \dots, S$ (có thể thêm chỉ số) để ký hiệu các ký tự không kết thúc.
- ◇ Các ký tự thường $a, b, c, \dots$ (có thể thêm chỉ số) để ký hiệu các ký tự kết thúc.
- ◇ Các ký tự in hoa X, Y, Z (có thể thêm chỉ số) để ký hiệu các ký tự kết thúc và không kết thúc.
- ◇ Các ký tự thường u, v, x, y, z (có thể thêm chỉ số) biểu thị các xâu gồm toàn ký hiệu kết thúc.
- ◇ Các ký tự Hy Lạp $\alpha, \beta, \gamma, \dots$ (có thể thêm chỉ số) biểu thị các xâu bất kỳ.

Ví dụ 2.2.5. Cho văn phạm $G = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aA, A \rightarrow bS, A \rightarrow a\})$.

a) Viết một dẫn xuất và xâu thuộc dẫn xuất.

b) Chỉ ra xâu thuộc $L(G)$ và xâu dạng chuẩn.

- Xâu

- Xâu

c) Tìm $L(G)$

Văn phạm G có thể xác định đầy đủ $L(G)$ nhưng không dễ dàng có được sự mô tả một cách chính xác về ngôn ngữ sinh bởi G . Tuy nhiên trong ví dụ này, kết quả là khá rõ ràng.

$$L(G) = \{(ab)^n a^2 \mid n \geq 0\} = \{ab\}^* \{a^2\}$$

Chúng ta có thể chứng minh kết quả này một cách dễ dàng. Nếu chúng ta nhận thấy hai quy tắc $S \rightarrow aA, A \rightarrow bS$ là đệ quy thì chúng ta có thể chứng minh bằng quy nạp toán học. Chúng ta sẽ chỉ ra rằng trong dãy dẫn xuất: $S \Rightarrow u_1 \Rightarrow u_2 \Rightarrow u_3 \Rightarrow u_4 \Rightarrow \dots \Rightarrow u_{2n-1} \Rightarrow u_{2n}$.

thì các xâu $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$ với $i = \overline{0, n}$ và $n \in \mathbb{N}$.

Bước cơ sở. Với $i = 0$ thì

- $S = \lambda S = (ab)^0 S$ nên $u_0 = (ab)^0 S$.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- $S \Rightarrow aA = \lambda aA = (ab)^0 aA$ nên $u_1 = (ab)^0 aA$.
- $S \Rightarrow aA \Rightarrow abS = (ab)^1 S$ nên $u_2 = (ab)^1 S$.

Bước quy nạp. Giả sử đẳng thức đúng đến $k = i$ ($0 \leq i < n$), tức là $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$, ta cần chứng minh đẳng thức đúng với $k = i + 1$.

$$u_{2i+2} = (ab)^{i+1} S \Rightarrow (ab)^{i+1} aA \Rightarrow (ab)^{i+1} abS = (ab)^{i+2} S$$

nên $u_{2i+3} = (ab)^{i+1} aA$ hay $u_{2(i+1)+1} = (ab)^{i+1} aA, u_{2i+4} = (ab)^{i+2} S$ hay $u_{2(i+2)} = (ab)^{i+2} S$.

Vậy đẳng thức đúng với $k = i + 1$ nên các xâu $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$ với $i = \overline{0, n}$. Từ đó suy ra $S \Rightarrow u_{2n} = (ab)^n S \Rightarrow (ab)^n aA \Rightarrow (ab)^n aa = (ab)^n a^2$.

Để đơn giản ta có thể viết như sau:

S

hay S

d) Từ văn phạm G tìm văn phạm sinh ra ngôn ngữ $L_1 = \{a(ab)^n a^2 | n \geq 0\}$.

Ý tưởng là ta sẽ phát triển G bằng cách tạo thêm ký hiệu a ở tiền tố của các xâu $L(G)$. Việc này có thể thực hiện được bằng cách thực hiện với sản xuất $S \rightarrow aA$ và những sản xuất khác có thể được chọn để A có thể suy ra ngôn ngữ trong đề bài. Khi đó ta có văn phạm:

Để

e) Chứng minh văn phạm $G_2 = (\{S_2, A_2, B_2\}, \{a, b\}, S_2, \{S_2 \rightarrow A_2 a, A_2 \rightarrow B_2 a | a, B_2 \rightarrow A_2 b\})$ tương đương với văn phạm G .

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

mà

Vậy

f) G là văn phạm thuộc nhóm nào?

Các

Ví dụ 2.2.6. Cho ngôn ngữ $L = \{01^n 0^{m+2} 1^2 | n, m \geq 0\}$.

a) Tìm văn phạm G sinh ra L .

Xâu

vì

b) Tìm văn phạm G_1 tương đương với văn phạm G .

Xâu

vì

Vậy

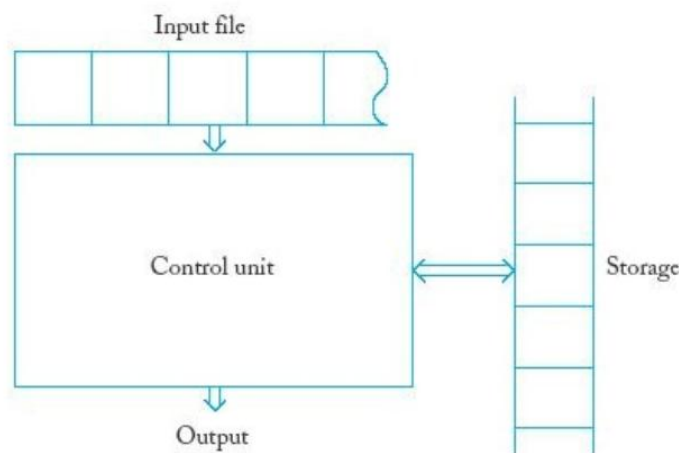
c) G là văn phạm thuộc nhóm nào?

Các

Định nghĩa 2.2.7.

III. Automata

Một automata (máy tự động) là một mô hình trừu tượng của máy tính, vì vậy, mọi automata đều bao gồm một số tính năng cần thiết. Nó có một thiết bị để đọc đầu vào với giả thiết rằng tập đầu vào là một xâu trên bảng chữ nhất định và được viết trên một băng vào nhưng automata chỉ có thể đọc và không thể thay đổi. Xâu đầu vào được chia vào các ô, mỗi ô chứa một ký hiệu. Automata đọc xâu vào từ trái sang phải, mỗi lần một ký hiệu và có cách thức để phát hiện điểm kết thúc của mỗi xâu vào (bằng cách cảm nhận điều kiện kết thúc của xâu vào). Automata cũng có thể tạo ra đầu ra ở một vài dạng. Automata có thể có một thiết bị lưu trữ tạm thời, bao gồm một số ô nhớ không giới hạn, mỗi ô chứa một ký hiệu từ một bảng chữ (có thể khác với bảng chữ ở xâu vào). Automata có thể đọc và thay đổi nội dung trên các ô nhớ. Cuối cùng, automata có một bộ điều khiển mà ở mỗi thời điểm có thể thuộc vào một trong hữu hạn các trạng thái và nó có thể thay đổi các trạng thái theo một cách xác định. Hình dưới đây biểu diễn một sơ đồ một automata tổng quát:



Một automata được giả thiết hoạt động trong các khoảng thời gian biệt lập. Tại mỗi thời điểm, automata ở một trạng thái nhất định, và đầu đọc đọc một ký hiệu trên xâu vào. Trạng thái tiếp theo của bộ điều khiển được xác định bởi hàm chuyển trạng thái. Hàm chuyển trạng thái xác định trạng thái tiếp theo của bộ điều khiển dựa vào ký hiệu đọc được trên xâu vào, trạng thái hiện tại của bộ điều khiển và thông tin đang được lưu trữ trên bộ nhớ.

Trong quá trình chuyển trạng thái, bộ điều khiển có thể tạo ra đầu ra hoặc thay đổi thông tin trên bộ nhớ. Thuật ngữ “cấu hình” sẽ được sử dụng để chỉ trạng thái tức thời của bộ điều khiển, xâu vào và bộ nhớ hiện thời. Sự chuyển đổi của automata từ cấu hình này sang cấu hình khác gọi là một “bước chuyển”.

Mô hình chung này bao gồm tất cả các automata mà ta nghiên cứu trong học phần này. Bộ điều khiển gồm hữu hạn các trạng thái là phổ biến với mọi trường hợp nhưng sự khác biệt giữa các automata đến từ phương thức tạo ra đầu ra và đặc tính của bộ lưu trữ tạm thời. Như chúng ta sẽ học, đặc tính của bộ lưu trữ tạm thời sẽ chi phối sức mạnh của các automata khác nhau.

Trong các phần tiếp theo, ta cần phân biệt giữa automata đơn định và automata không đơn định. Một automata đơn định là tại mỗi “bước chuyển” xác định một cách duy nhất bởi cấu hình hiện

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

tại. Nếu chúng ta biết trạng thái hiện tại của automata, ký hiệu trên xâu vào đang được đọc và bộ nhớ hiện tại thì chúng ta có thể dự đoán chính xác cấu hình tiếp theo của automata. Trong một automata không đơn định, điều này là không phải như vậy. Tại mỗi thời điểm, automata không đơn định có thể có một số bước chuyển khả thi, vì vậy chúng ta chỉ có thể dự đoán một tập hợp các cấu hình tiếp theo có thể có của automata. Mỗi quan hệ giữa automata đơn định và automata không đơn định đóng vai trò quan trọng trong nghiên cứu của chúng ta.

Một automata có đầu ra ở mức giới hạn đơn giản là “có” hoặc “không” được gọi là một “accepter”. Với một xâu vào, “accepter” sẽ đoán nhận xâu vào hoặc không đoán nhận xâu vào. Một automata tổng quát hơn, có khả năng tạo ra các ký hiệu đầu ra gọi là một “transducer”.

§3. PHẦN LOẠI VĂN PHẠM THEO CHOMSKY

I. Văn phạm tổng quát

1. Định nghĩa và ví dụ

Định nghĩa 3.1.1. Văn phạm $G = (V, T, S, P)$ được gọi là văn phạm tổng quát hay văn phạm không hạn chế hay văn phạm nhóm 0 nếu không có bất kỳ một ràng buộc nào đối với tập sản xuất P .

- *Ngôn ngữ do văn phạm tổng quát sinh ra gọi là ngôn ngữ tổng quát.*

Ví dụ 3.1.2. Trong hóa học tính toán hoặc sinh học phân tử (DNA computing): Các sợi DNA có thể kết hợp, tách ra và biến đổi dựa trên các enzyme. Các quy tắc biến đổi DNA chính là các quy tắc của văn phạm tổng quát.

Ví dụ: Một chuỗi gen AGT khi gặp enzyme X sẽ biến thành TCA. Đây là quy tắc $\alpha \rightarrow \beta$ không giới hạn độ dài, đặc trưng của loại 0.

Ví dụ 3.1.3. (Hệ thống phân loại và đóng gói thông minh). Giả sử bạn có một dây chuyền gồm các sản phẩm chạy hỗn loạn, và bạn cần một hệ thống có thể:

- *Đếm số lượng sản phẩm.*
- *Kiểm tra tính đối xứng (ví dụ: bộ linh kiện phải đi kèm nhau).*
- *Sắp xếp lại vị trí linh kiện ngay trên băng chuyền.*

Trong khi các văn phạm cấp thấp hơn chỉ có thể "đếm" hoặc "nhận diện" theo một chiều, văn phạm tổng quát cho phép bạn thực hiện quy tắc:

"Nếu thấy A đứng trước B, hãy đổi chỗ chúng và biến B thành C"

2. Ý nghĩa của văn phạm tổng quát

Ý nghĩa lớn nhất của văn phạm tổng quát nằm ở sức mạnh biểu diễn:

- *Tính toán được (Computability): Một ngôn ngữ được sinh ra bởi văn phạm tổng quát nếu và chỉ nếu nó được chấp nhận bởi một máy Turing. Điều này có nghĩa là bất kỳ*

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

thuật toán nào con người có thể viết ra đều có thể biểu diễn dưới dạng một văn phạm tổng quát.

- *Giới hạn cuối cùng: Nó đại diện cho ranh giới của những gì có thể mô tả được bằng toán học và logic trong ngôn ngữ. Nếu một ngôn ngữ không thể biểu diễn bằng văn phạm tổng quát, nó được coi là "không thể tính toán" (uncomputable).*

3. Ứng dụng của văn phạm tổng quát

Văn phạm tổng quát rất khó để triển khai trong thực tế (vì bài toán kiểm tra xem một chuỗi có thuộc ngôn ngữ này hay không là *không thể quyết định được* - Undecidable), nó vẫn có các ứng dụng quan trọng:

- ✦ *Lý thuyết Ngôn ngữ lập trình:* Dùng để nghiên cứu giới hạn của các trình biên dịch (compilers). Nó giúp các nhà khoa học máy tính hiểu được những cấu trúc ngôn ngữ nào là quá phức tạp để máy tính có thể xử lý hiệu quả.
- ✦ *Trí tuệ nhân tạo (AI) cổ điển:* Trong các hệ thống dựa trên quy tắc (Rule-based systems), văn phạm tổng quát mô phỏng cách tri thức được biến đổi và suy diễn.
- ✦ *Sinh học tính toán:* Mô hình hóa các chuỗi DNA phức tạp và sự tái tổ hợp di truyền, nơi các đoạn mã có thể di chuyển, đảo ngược hoặc thay đổi dựa trên ngữ cảnh xung quanh.
- ✦ *Xử lý ngôn ngữ tự nhiên (NLP):* Mặc dù ngôn ngữ tự nhiên (tiếng Việt, tiếng Anh) thường được coi là nằm giữa Loại 1 và Loại 2, văn phạm tổng quát cung cấp khung lý thuyết để giải thích các hiện tượng ngôn ngữ cực kỳ phức tạp mà các mô hình đơn giản không làm được.

Mặc dù nó "vạn năng", nhưng văn phạm tổng quát có một nhược điểm chí mạng: *Bài toán dừng (The Halting Problem)*. Nếu viết một chương trình dựa hoàn toàn trên văn phạm tổng quát, không thể chắc chắn 100% bằng toán học rằng chương trình đó sẽ chạy xong hay sẽ bị treo (lặp vô hạn) trừ khi bạn chạy nó. Do đó, trong thực tế, các ngôn ngữ lập trình thường cố gắng giới hạn ở các cấp độ thấp hơn để đảm bảo tính ổn định.

II. Văn phạm cảm ngữ cảnh

1. Định nghĩa và ví dụ

Định nghĩa 3.2.1.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

•

Ví dụ 3.2.2. Cho văn phạm $G = (\{S, A, B\}, \{0, 1, 2\}, S, P)$ với tập sản xuất:

$$P = \{S \rightarrow 0SAB|01B, BA \rightarrow AB, A \rightarrow 1, B \rightarrow 2\}$$

a) Chứng minh văn phạm G là văn phạm cảm ngữ cảnh.

Do

b) Tìm một xâu thuộc $L(G)$.

Vậy

2. Ý nghĩa của văn phạm cảm ngữ cảnh

- **Sức mạnh trung gian:** Nó mạnh hơn văn phạm phi ngữ cảnh (dùng trong lập trình) nhưng yếu hơn văn phạm tổng quát (máy Turing). Nó có thể mô tả hầu hết các cấu trúc toán học và ngôn ngữ tự nhiên phức tạp.
- **Tính quyết định (Decidability):** Khác với văn phạm tổng quát (Loại 0), văn phạm cảm ngữ cảnh có tính quyết định. Nghĩa là luôn có một thuật toán để kiểm tra xem một chuỗi cho trước có thuộc ngôn ngữ đó hay không (dù tốn thời gian).
- **Mô hình máy tương ứng:** Ngôn ngữ cảm ngữ cảnh được chấp nhận bởi máy Turing giới hạn biên (Linear Bounded Automaton - LBA). Đây là một máy Turing mà bộ nhớ (băng ghi) bị giới hạn bởi độ dài của chuỗi đầu vào.

3. Ứng dụng thực tế

Dù ít được nhắc đến hơn văn phạm chính quy hay phi ngữ cảnh trong lập trình hằng ngày, văn phạm cảm ngữ cảnh lại cực kỳ quan trọng trong các lĩnh vực chuyên sâu:

- ✱ Ngôn ngữ tự nhiên (như tiếng Việt, tiếng Anh) thực chất là "cảm ngữ cảnh".

Ví dụ 3.2.3. Trong tiếng Việt, từ "đường" có thể là "con đường" hoặc "đường ăn". Để hiểu nghĩa của nó, máy tính cần nhìn vào các từ xung quanh (ngữ cảnh). Các mô hình ngôn ngữ hiện đại (như Transformer/GPT) thực chất là các bộ máy xử lý ngữ cảnh cực kỳ phức tạp để giải quyết vấn đề này.

- ✱ Kiểm tra kiểu trong lập trình (Type Checking): Hầu hết các ngôn ngữ lập trình (C++, Java, Python) không hoàn toàn là "phi ngữ cảnh".

Ví dụ 3.2.4. Bạn không thể sử dụng biến x nếu chưa khai báo nó trước đó. Việc kiểm tra xem "biến đã khai báo chưa" là một bài toán cảm ngữ cảnh. Trình biên dịch phải nhìn lại phần "ngữ cảnh" phía trên của mã nguồn để xác định tính hợp lệ của dòng hiện tại.

- ✱ Sinh học phân tử (Bioinformatics): Cấu trúc bậc hai của RNA hoặc các tương tác giữa các protein thường có dạng cảm ngữ cảnh. Các đoạn mã di truyền không hoạt động độc lập; chúng thay đổi chức năng dựa trên các đoạn mã nằm liền kề hoặc cách một khoảng xác định (long-range dependencies).
- ✱ Trong trình biên dịch (Compilers): Vì việc xử lý trực tiếp bằng văn phạm cảm ngữ cảnh (Loại 1) rất tốn kém về tài nguyên tính toán (độ phức tạp cực lớn), các kỹ sư máy tính thường dùng mẹo:
 - Sử dụng văn phạm phi ngữ cảnh (CFG) để kiểm tra cấu trúc lệnh (ngoặc đơn, dấu chấm phẩy...).
 - Sử dụng bảng ký hiệu (Symbol Table) để xử lý phần "cảm ngữ cảnh". Bảng này lưu trữ mọi biến đã khai báo. Khi trình biên dịch thấy một biến, nó tra cứu trong bảng (đây chính là hành động kiểm tra ngữ cảnh).

Nếu không có tính chất "cảm ngữ cảnh", máy tính sẽ không bao giờ hiểu được mối quan hệ giữa các phần khác nhau trong một chương trình, và chúng ta sẽ không có các trình biên dịch thông minh như ngày nay.

III. Văn phạm phi ngữ cảnh

Văn phạm phi ngữ cảnh (Context – Free Grammar – CFG) là loại văn phạm quan trọng nhất và được ứng dụng rộng rãi nhất trong khoa học máy tính, đặc biệt là trong việc thiết kế các ngôn ngữ lập trình.

1. Định nghĩa và ví dụ

Định nghĩa 3.3.1.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

•

Ví dụ 3.3.2. Cho văn phạm: $G = (\{S, A, B\}, \{0, 1, 2\}, S, \{S \rightarrow S2 \mid A2, A \rightarrow 0A1 \mid 01\})$

a) Chứng minh rằng G là văn phạm phi ngữ cảnh.

Về trái

b) Tìm $L(G)$.

Vậy

Ví dụ 3.3.3. Cho ngôn ngữ $L = \{aba^{n+1}b^{n+2}a^{m+3} \mid n, m \geq 0\}$.

a) Tìm văn phạm G sinh ra L .

Xâu

vì

S

Với

b) G là văn phạm thuộc nhóm nào?

Tập

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

-

-

Vậy G

Ví dụ 3.3.4. Cho văn phạm: $G = (\{S\}, \{a, b\}, S, \{S \rightarrow aSa|bSb|aa|bb|\lambda\})$

a) Chứng minh rằng G là văn phạm phi ngữ cảnh.

Tập

-

-

Vậy G

b) Viết 7 xâu khác nhau độ dài lớn hơn 8 khác nhau sinh bởi G .

c) **Tìm $L(G)$.**

Ta sẽ chứng minh $L(G) = L = \{uu^R | u \in \{a, b\}^*\}$.

Chiều thuận. Chứng minh $L(G) \subseteq L$, tức là mọi xâu $v \in L(G)$ thì $v \in L$, tức là $v = uu^R$ với $u \in \{a, b\}^*$. Ta sẽ chứng minh bằng quy nạp theo số lần dẫn xuất từ S .

Gọi n là số lần dẫn xuất từ S đến xâu kết thúc $v \in L(G)$.

Bước cơ sở. Với $n = 1$ ta có:

- $S \Rightarrow \lambda = \lambda\lambda^R \in L(G)$ mà $\lambda\lambda^R \in L$.
- $S \Rightarrow aa = aa^R \in L(G)$ mà $aa^R \in L$.
- $S \Rightarrow bb = bb^R \in L(G)$ mà $bb^R \in L$.

Vậy bài toán đúng với số lần dẫn xuất là 1 từ S để đến được xâu kết thúc thuộc $L(G)$ và xâu này cũng thuộc L .

Bước quy nạp. Giả sử sau n ($n \geq 1$) lần suy dẫn ta có $S \Rightarrow^* v$ với $v = uu^R$. Ta cần chứng minh sau $n + 1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a, b\}^*$.

Trường hợp 1. Lần suy dẫn đầu tiên ta sử dụng sản xuất $S \Rightarrow aSa$. Khi đó từ S thuộc xâu aSa , sau n lần suy diễn ta được xâu $v = uu^R \in L(G)$ với $u \in \{a, b\}^*$. Khi đó

$$S \Rightarrow aSa \Rightarrow^* auu^Ra = auu^Ra^R = (au)(ua)^R$$

mà $au \in L(G)$ nên sau $n + 1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a, b\}^*$.

Trường hợp 2. Lần suy dẫn đầu tiên ta sử dụng sản xuất $S \Rightarrow bSb$. Khi đó từ S thuộc xâu bSb , sau n lần suy diễn ta được xâu $v = uu^R \in L(G)$ với $u \in \{a, b\}^*$. Khi đó

$$S \Rightarrow bSb \Rightarrow^* buu^Rb = buu^Rb^R = (bu)(ub)^R$$

mà $bu \in L(G)$ nên sau $n + 1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a, b\}^*$.

Theo nguyên lý quy nạp toán học, sau n lần suy diễn bất kỳ từ S , ta đến được các xâu v có dạng uu^R với $u \in \{a, b\}^*$, tức là $v = uu^R \in L$.

Vậy $L(G) \subseteq L$.

Chiều đảo. Ta sẽ chứng minh $L \subseteq L(G)$, tức là mọi xâu $v = uu^R \in L$ với $u \in \{a, b\}^*$ thì $uu^R \in L(G)$. Ta sẽ chứng minh bằng chiều dài của xâu uu^R .

Đặt $2n = |v| = |uu^R|$ với $u \in \{a, b\}^*$.

Bước cơ sở:

- Với $n = 0$ thì $v = \lambda = \lambda\lambda^R \in L$ mà $S \Rightarrow \lambda$ nên $\lambda \in L(G)$
- Với $n = 1$ thì
 - + Nếu $v = aa = aa^R \in L$ mà $S \Rightarrow aa$ nên $aa \in L(G)$.
 - + Nếu $v = bb = bb^R \in L$ mà $S \Rightarrow bb$ nên $bb \in L(G)$.

Bước quy nạp. Giả sử xâu $v \in L$ với $|v| = 2n$ thì $v \in L(G)$. Ta sẽ chứng minh nó cũng đúng với các xâu $v \in L$ với $|v| = 2(n + 1)$.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Do $v \in L$ và $|v| = 2(n+1)$ nên $v = av_1a$ hoặc $v = bv_1b$ với $v_1 \in L$ và $|v_1| = 2n$. Khi đó theo giả thiết quy nạp, $v_1 \in L(G)$ nên $S \Rightarrow^* v_1$.

Trường hợp 1. $v = av_1a$ thì $S \Rightarrow aSa \Rightarrow^* av_1a = v$ nên $v \in L(G)$.

Trường hợp 2. $v = bv_1b$ thì $S \Rightarrow bSb \Rightarrow^* bv_1b = v$ nên $v \in L(G)$.

Theo nguyên lý quy nạp toán học, xâu $v \in L$ mà $|v| = 2(n+1)$ thì $v \in L(G)$ nên $L \subseteq L(G)$.

Vậy $L = L(G)$ hay G sinh ra ngôn ngữ $\{uu^R | u \in \{a, b\}^*\}$.

Ví dụ 3.3.5. Xét xem đoạn chương trình dưới đây thuộc lớp văn phạm cảm ngữ cảnh hay phi ngữ cảnh:

Program Test;

var x: integer;

Begin

x := 10; {Hợp lệ}

y := 5; {Lỗi: y chưa được khai báo}

End;

- Nếu là chương trình thuộc lớp văn phạm phi ngữ cảnh: Trình biên dịch chỉ kiểm tra xem cấu trúc câu lệnh $y := 5$; có đúng dạng *biến* $:=$ *giá trị*; hay không. Nó không quan tâm y là ai, vì nó không có khả năng "nhìn lại" hoặc "nhìn xung quanh" để kiểm tra danh sách biến đã khai báo.
- Nếu chương trình thuộc lớp văn phạm cảm ngữ cảnh: Trình biên dịch buộc phải kiểm tra ngữ cảnh (phần khai báo *var* phía trên). Câu lệnh $y := 5$ chỉ được coi là "đúng ngữ pháp" nếu trong ngữ cảnh trước đó đã xuất hiện chuỗi *var y*.

2. Ý nghĩa của văn phạm phi ngữ cảnh

- **Cấu trúc phân cấp:** CFG có khả năng mô tả các cấu trúc lồng nhau vô hạn. Điều này giúp nó vượt xa văn phạm chính quy.
- **Mô hình máy tương ứng:** Ngôn ngữ phi ngữ cảnh được chấp nhận bởi automata đẩy xuống (Pushdown Automata - PDA). Đây là một automata hữu hạn được trang bị thêm một ngăn xếp (Stack) để ghi nhớ dữ liệu.
- **Tính hiệu quả:** Các thuật toán phân tích cú pháp (parsing) cho CFG (như thuật toán CYK hoặc Earley) rất hiệu quả, cho phép máy tính xử lý các tệp mã nguồn khổng lồ trong thời gian ngắn.

3. Ứng dụng của văn phạm phi ngữ cảnh

Văn phạm phi ngữ cảnh là "xương sống" của hầu hết các công nghệ xử lý ngôn ngữ hiện nay.

- ✦ **Thiết kế trình biên dịch (Compilers):** Hầu hết cú pháp của các ngôn ngữ lập trình như C, Java, Python được định nghĩa bằng CFG (sử dụng ký pháp Backus-Naur Form).

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 3.3.6. Cấu trúc các cặp ngoặc lồng nhau: $If ((a > b) \ \&\& (c < d))$

Trình biên dịch sử dụng Stack để đảm bảo mọi dấu ngoặc mở đều có dấu ngoặc đóng tương ứng.

- ✱ Trình phân tích cú pháp (Parser): Các công cụ nổi tiếng như **Yacc** hay **Bison** được dùng để tạo ra trình biên dịch cho các ngôn ngữ như C, C++, hay SQL thực chất là những cỗ máy thực thi các quy tắc của văn phạm phi ngữ cảnh.
- ✱ Phân tích cấu trúc dữ liệu (JSON, XML, HTML): Các định dạng dữ liệu hiện đại đều có cấu trúc cây (tree-based). Trong XML: $\langle tag \rangle \dots \langle /tag \rangle$. Thẻ đóng phải khớp với thẻ mở gần nhất. Đây chính là biểu hiện của một ngôn ngữ phi ngữ cảnh.
- ✱ Biểu thức toán học: Các máy tính bỏ túi xử lý các biểu thức như

$$(1 + 2) * \frac{3}{5 - 1}$$

dựa trên các quy tắc phi ngữ cảnh để xác định thứ tự ưu tiên của các phép toán.

- ✱ Trình soạn thảo văn bản (Code Editor): Các tính năng như "*Highlight matching brackets*" (tô sáng cặp ngoặc tương ứng) trong VS Code hay Sublime Text hoạt động dựa trên logic của văn phạm phi ngữ cảnh.
- ✱ Ngôn ngữ học : CFG được sử dụng để phân tích cấu trúc cú pháp của câu trong ngôn ngữ tự nhiên ($S \rightarrow NP VP$ - Chủ ngữ + Vị ngữ). Mặc dù ngôn ngữ tự nhiên có những yếu tố "cảm ngữ cảnh", nhưng CFG vẫn là công cụ nền tảng để sơ đồ hóa câu văn.

IV. Văn phạm chính quy

Văn phạm chính quy (Regular Grammar) là cấp độ thấp nhất nhưng lại là cấp độ được sử dụng phổ biến nhất trong thực tế hằng ngày của một lập trình viên. Nó đại diện cho những ngôn ngữ có cấu trúc đơn giản, lặp đi lặp lại và không có tính chất lồng nhau phức tạp.

1. Văn phạm chính quy

Định nghĩa 3.4.1.

•

•

•

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

•

Chú ý. Trong văn phạm chính quy, có nhiều nhất một ký hiệu không kết thúc nằm ở vế phải của tất cả các sản xuất. Hơn nữa ký hiệu không kết thúc phải nhất quán hoặc là ký hiệu bên trái hoặc là ký hiệu bên phải của tất cả các sản xuất.

Ví dụ 3.4.2. Cho văn phạm $G = (\{S, A, B\}, \{0, 1, 2\}, S, \{S \rightarrow 0S \mid 0A, A \rightarrow 1A \mid 1B, B \rightarrow 2B \mid 2\})$.

a) Chứng minh văn phạm G là văn chính quy.

Các

b) Tìm $L(G)$.

S

Vậy

Với

Ví dụ 3.4.3.

a) Xây dựng văn phạm G trên bảng chữ nhị phân sinh ra các xâu có tiền tố 01, chứa xâu con 101 và có hậu tố là 00.

G

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$G =$

thỏa mãn

S

Với

b) Chứng minh G là văn phạm chính quy.

Các

c) Xây dựng FA M_1 tương đương với G .

Do

Do

Do

Do

Do

Do

Do

Do

Do

Vậy FA

d) Xây dựng DFA M_2 tương đương với G .

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

δ'	0	1

Vậy

2. Văn phạm tuyến tính

Định lý 3.4.4. Nếu $G = (V, T, S, P)$ là văn phạm tuyến tính trái thì tồn tại văn phạm tuyến tính phải tương đương với G .

Xây dựng văn phạm tuyến tính phải $G' = (V, T, S, P')$ như sau: Nếu $A \rightarrow Bu$ và $A \rightarrow u$ ($A, B \in V, u \in T^*$) là các sản xuất trong P thì thêm vào P' các sản xuất $A \rightarrow u^R B$ và $A \rightarrow u$. Khi đó có thể chứng minh được $L(G') = L(G)$.

Ví dụ 3.4.5. Văn phạm $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow abA|Bb, A \rightarrow Aba|aB, B \rightarrow a|\lambda\})$ không là văn phạm chính quy mặc dù các sản xuất của G hoặc có dạng tuyến tính trái, hoặc có dạng tuyến tính phải nhưng G không ở dạng tuyến tính trái cũng như tuyến tính phải.

Các

Chú ý. Một văn phạm tuyến tính là một văn phạm có nhiều nhất một ký hiệu không kết thúc nằm ở vế phải của tất cả các sản xuất và không hạn chế vị trí của biến này.

- ✓ Một văn phạm chính quy luôn là văn phạm tuyến tính nhưng văn phạm tuyến tính chưa chắc đã là văn phạm chính quy.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

3. Một số ngôn ngữ chính quy

Ví dụ 3.4.6. Cho bảng chữ $\Sigma = \{a_1, a_2, \dots, a_n\}$. Chứng minh các ngôn ngữ sau đều là các ngôn ngữ chính quy:

a) $L_1 = \{a_1 a_2 \dots a_n\}$.

Văn phạm

nên

b) $L_2 = \Sigma^*$

Văn phạm

nên

c) $L_3 = \Sigma^+$

Văn phạm

- Nếu

- Nếu

nên

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

d) $L_4 = \emptyset$

Văn phạm

nên

Ví dụ 3.4.6. Một địa chỉ email thường có dạng $Username@Domain.Extension$ với $Username, Domain, Extension$ là các xâu gồm các chữ cái tiếng Anh (cả chữ hoa và chữ thường), chữ số và có thể thêm các dấu ".", "_", các ký hiệu "%", "-", "+", "*", "/", "=", "!", "#", "\$", "&", "'", "?", "^", "{", "|", "}", "~".

a) Xây dựng văn phạm G sinh ra địa chỉ email. Chứng minh G là văn phạm chính quy.

Văn phạm $G = (\{S, Username, Domain, Extension\}, \Sigma, S, P)$ với Σ là tập các ký tự trong địa chỉ email có thể có, P là tập sản xuất có dạng:

$$S \rightarrow [ký\ tự]S|@Domain$$

$$Domain \rightarrow [ký\ tự]Domain|.Extension$$

$$Extension \rightarrow [ký\ tự]Extension|[ký\ tự]Extension_1$$

$$Extension_1 \rightarrow [ký\ tự]$$

Khi đó một email là $hoa12@huce.edu.vn$ sẽ được văn phạm G sinh ra như sau:

$$\begin{aligned} S &\Rightarrow hS \Rightarrow hoS \Rightarrow hoaS \Rightarrow hoa1S \Rightarrow hoa12S \Rightarrow hoa12@Domain \Rightarrow hoa12@hDomain \Rightarrow \\ &hoa12@huDomain \Rightarrow hoa12@hucDomain \Rightarrow hoa12@huceDomain \Rightarrow \\ &hoa12@huce.Extension \Rightarrow hoa12@huce.eExtension \Rightarrow hoa12@huce.edExtension \Rightarrow \\ &hoa12@huce.eduExtension \Rightarrow hoa12@huce.edu.Extension \Rightarrow \\ &hoa12@huce.edu.vExtension_1 \Rightarrow hoa12@huce.edu.vn \end{aligned}$$

Các sản xuất trong P đều có dạng tuyến tính phải nên G là văn phạm chính quy.

b) Xây dựng FA M sinh ra địa chỉ email.

FA $M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \Sigma, q_0, \delta, \{q_6\})$ với Σ là tập các ký tự trong địa chỉ email có thể có với hàm chuyển trạng thái δ như sau:

$$\delta(q_0, [ký\ tự]) = \{q_1\}$$

$$\delta(q_1, [ký\ tự]) = \{q_1\}$$

$$\delta(q_1, @) = \{q_2\}$$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$$\delta(q_2, [k\acute{y} t\grave{u}]) = \{q_3\}$$

$$\delta(q_3, [k\acute{y} t\grave{u}]) = \{q_3\}$$

$$\delta(q_3, \cdot) = \{q_4\}$$

$$\delta(q_4, [k\acute{y} t\grave{u}]) = \{q_4, q_5\}$$

$$\delta(q_5, [k\acute{y} t\grave{u}]) = \{q_6\}$$

Khi đó một email là `hoa12@huce.edu.vn` sẽ FA M đoán nhận như sau:

$q_0\text{hoa12@huce.edu.vn} \Rightarrow q_1\text{oa12@huce.edu.vn} \Rightarrow q_1\text{a12@huce.edu.vn} \Rightarrow$
 $q_1\text{12@huce.edu.vn} \Rightarrow q_1\text{2@huce.edu.vn} \Rightarrow q_1\text{@huce.edu.vn} \Rightarrow q_2\text{huce.edu.vn} \Rightarrow$
 $q_3\text{uce.edu.vn} \Rightarrow q_3\text{ce.edu.vn} \Rightarrow q_3\text{e.edu.vn} \Rightarrow q_3\text{.edu.vn} \Rightarrow q_4\text{edu.vn} \Rightarrow q_4\text{du.vn} \Rightarrow$
 $q_4\text{u.vn} \Rightarrow q_4\text{.vn} \Rightarrow q_4\text{vn} \Rightarrow q_5\text{n} \Rightarrow q_6 \in \{q_6\} = F$

Ứng dụng thực tế của mô hình này

Khi bạn hiểu được sơ đồ trạng thái, bạn sẽ biết cách tối ưu Regex:

- ✖ Hiệu năng: DFA chạy cực nhanh vì nó không bao giờ phải quay lui (backtracking) nếu bạn viết Regex tốt.
- ✖ Bảo mật: Tránh được lỗi ReDoS (Regular Expression Denial of Service) – lỗi xảy ra khi Regex quá phức tạp khiến máy tính tốn quá nhiều trạng thái trung gian để xử lý một chuỗi đầu vào độc hại.

4. Ý nghĩa của văn phạm chính quy

- Tính hữu hạn: Nó tương đương với automata hữu hạn (Finite Automata - FA). Điều này có nghĩa là máy tính chỉ cần một lượng bộ nhớ cố định và cực kỳ ít để nhận diện ngôn ngữ này. Nó không cần ngăn xếp (stack) như văn phạm cảm ngữ cảnh.
- Tốc độ cực nhanh: Vì cấu trúc đơn giản, việc kiểm tra một chuỗi có chính quy hay không diễn ra gần như tức thời, với độ phức tạp thời gian là tuyến tính $O(n)$.
- Biểu thức chính quy (Regex): Đây là công cụ cụ thể hóa văn phạm chính quy vào lập trình thực tế.

5. Ứng dụng của văn phạm chính quy

- ✖ Biểu thức chính quy (Regular Expressions - Regex): Đây là ứng dụng nổi tiếng nhất. Mọi ngôn ngữ lập trình (Python, JS, Java...) đều có thư viện Regex để:
 - Kiểm tra định dạng email.
 - Kiểm tra số điện thoại: Đảm bảo chuỗi chỉ chứa các chữ số và có độ dài đúng quy định.
 - Tìm kiếm và thay thế: Tìm tất cả các từ "Color" để đổi thành "Colour" trong một văn bản lớn.
 - Bộ phân tích từ vựng (Lexical Analyzer / Scanner): Trong trình biên dịch, trước khi kiểm tra cú pháp phức tạp, máy tính sẽ dùng văn phạm chính quy để "quét" mã

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

nguồn và phân loại các thành phần cơ bản (Tokens) như: Từ khóa (if, else, while); Tên biến (Identifier); Các toán tử (+, -, *, /).

- ✱ Thiết kế phần cứng (Digital Logic Design): Các mạch điều khiển trong đèn giao thông, máy bán nước tự động, hay các bộ vi xử lý đơn giản hoạt động dựa trên máy tự động hữu hạn — mô hình vật lý của văn phạm chính quy.
- ✱ Tìm kiếm trong văn bản (Grep, Sed, Awk): Các công cụ dòng lệnh trên Linux sử dụng văn phạm chính quy để lọc dữ liệu từ các file log khổng lồ của hệ thống.

V. So sánh các văn phạm

1. Bảng so sánh các văn phạm

Tiêu chí	Chính quy	Phi ngữ cảnh	Cảm ngữ cảnh	Tổng quát
Mô hình máy	Automata hữu hạn	Automata đẩy xuống	Automata giới hạn biên	Máy Turing
Bộ nhớ	Không có bộ nhớ phụ (chỉ có trạng thái hữu hạn)	Ngăn xếp (Stack) - LIFO (Vào sau ra trước)	Bảng ghi giới hạn (Bảng độ dài chuỗi đầu vào)	Bảng ghi vô hạn (Có thể đọc/ghi tùy ý)
Quy tắc thể	$A \rightarrow aB$ hoặc $A \rightarrow a$	$A \rightarrow \alpha$	$\alpha_1 A \alpha_2 \rightarrow \beta$	$\alpha \rightarrow \beta$ ($ \alpha \geq 1$)
Cấu trúc đặc trưng	Chuỗi lặp, mẫu ký tự đơn giản.	Cấu trúc lồng nhau, đệ quy, đối xứng gương.	Cấu trúc phụ thuộc ngữ cảnh, sao chép chuỗi	Mọi cấu trúc có thể mô tả bằng thuật toán.
Khả năng	Nhận diện từ khóa, định dạng số, email	Phân tích cú pháp ngoặc	Kiểm tra khai báo biến, kiểm tra kiểu dữ liệu, ngôn ngữ tự nhiên.	Giải quyết mọi bài toán có thể tính toán được.
Giới hạn	Không thể đếm hoặc nhớ các cặp lồng nhau	Không thể khớp các cấu trúc song song	Độ phức tạp tính toán rất cao (NP-complete).	Bài toán dừng: Không thể chắc chắn máy sẽ dừng hay chạy mãi mãi.

2. Phân tích sâu các đặc điểm cốt lõi của văn phạm

a) Sự khác biệt về bộ nhớ (Cơ chế lưu trữ)

- *Văn phạm chính quy*: Giống như một người đi bộ không có giấy bút, chỉ nhớ mình đang ở ngã tư nào.
- *Văn phạm phi ngữ cảnh*: Có một chồng đĩa (Stack). Bạn có thể đặt thêm đĩa lên hoặc lấy đĩa trên cùng ra. Điều này giúp máy "nhớ" được mình đã mở bao nhiêu dấu ngoặc để đóng cho đúng.
- *Văn phạm cảm ngữ cảnh*: Có một đoạn giấy vừa đủ bằng độ dài bài toán. Bạn có thể viết đi viết lại trên đó để kiểm tra các mối liên hệ phức tạp.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- **Văn phạm tổng quát:** Có một cuộn giấy dài vô tận. Bạn có thể ghi chép, tính toán nháp và lưu trữ mọi thứ mà không sợ hết chỗ.

b) Cấu trúc đặc trưng và khả năng "đếm"

- **Văn phạm chính quy:** Chỉ có thể nhận diện các mẫu như "Một chuỗi các chữ số". Nó không thể so sánh số lượng giữa hai nhóm (ví dụ không thể biết số chữ số 0 và số chữ số 1 trong xâu có bằng nhau hay không).
- **Văn phạm phi ngữ cảnh:** Có thể đếm theo cặp (ví dụ số chữ số 0 bằng số chữ số 1 như trong xâu $0^n 1^n$).
- **Văn phạm cảm ngữ cảnh:** Có thể đếm bộ ba hoặc nhiều hơn (ví dụ $0^n 1^n 2^n$ hoặc kiểm tra xem một biến đã xuất hiện ở phần trên của chương trình chưa).

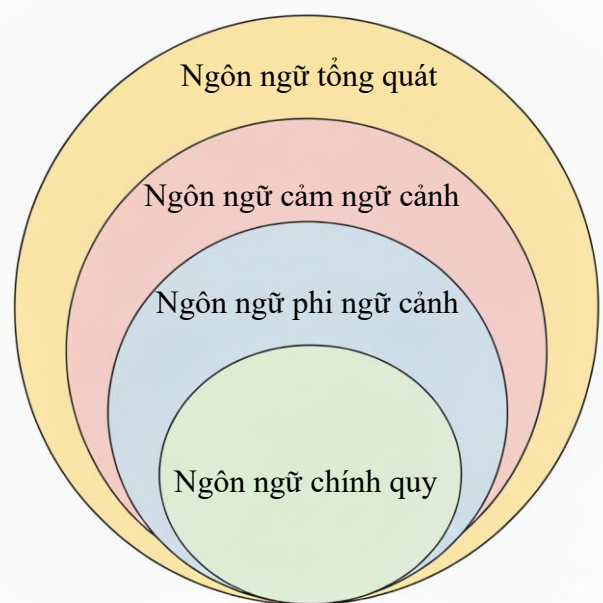
c) Các sản xuất

- Trong văn phạm phi ngữ cảnh, vế trái chỉ có một ký hiệu không kết thúc (A). Nó không quan tâm hàng xóm là ai
- Trong văn phạm cảm ngữ cảnh, vế trái có hàng xóm và phải chứa ký hiệu không kết thúc ($\alpha_1 A \alpha_2$). Nó chỉ thay đổi nếu có đúng hàng xóm thân thiết bao quanh.
- Trong văn phạm tổng quát, vế trái là bất cứ thứ gì. Sự biến đổi diễn ra hoàn toàn tự do.

Nhận xét. Lớp các ngôn ngữ chính quy

$\subset$ Lớp các ngôn ngữ phi ngữ cảnh $\subset$

Lớp các ngôn ngữ cảm ngữ cảnh $\subset$ Lớp các ngôn ngữ tổng quát.



BÀI TẬP BUỔI 2

Bài 1. Cho văn phạm $G =$

a) Cho G sinh ra xâu $u =$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

Bài 2. Cho văn phạm $G =$

a) Cho G sinh ra chuỗi $u =$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

Bài 3. Cho văn phạm $G =$

a) Cho G sinh ra chuỗi $u =$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN